Theatre Booking Project

George Cooke

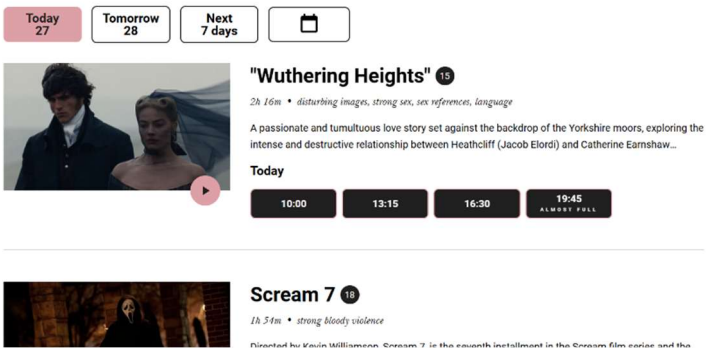
2026

Index:

Similar Programs & Inspiration

There are a multitude of different booking apps used around the world for all different purposes. Some are to book tickets to films, other for music festivals, some for sports, and some for aircraft seats. Going into this project the correlation that immediately jumped out to me was with booking tickets to the cinema. With the presence of assigned seating on a regular layout with showings that may not have predictable or consistent showing dates.

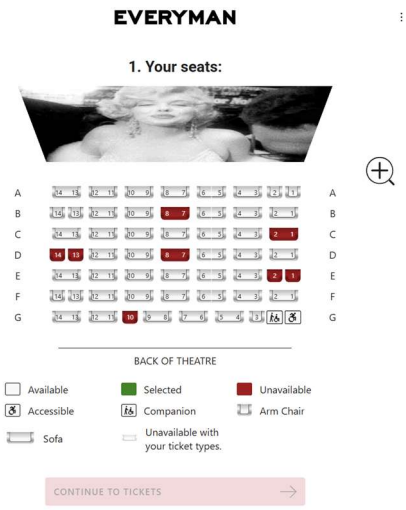
The programs I aim to take inspiration from are the Cineworld and Everyman cinemas. These are two good examples of seating booking systems that I have experience using and the layouts of these tools have been well tested and refined over the years that they have been in use.



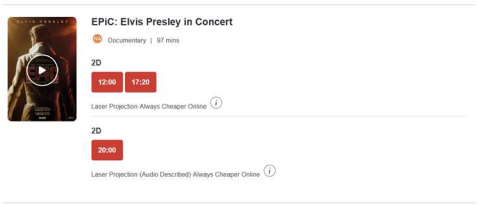
The image to the right is a screenshot of the Everyman website displaying the different films available. I like the possibility to go straight in to book specific dates and times for the films and will likely incorporate this as a target for my own program. Other than that, I like the presence of a description which is something I will copy for my own program. I will also take from this the horizontal line separating the different films. I believe that this will lead to a less cluttered appearance compared to my previous program with many boxes with rounded corners.

From the above page, a person can click on the title and go into further information and date/time selection. My program will assume that all showings are at the same time and there is a maximum of one showing per day per performance but there can be multiple performances at once in different rooms (it could be a stretch target to extend this to having separate time slots). I will likely replace this page with a custom one of my own design with just a number of simple buttons each with the date that they represent.

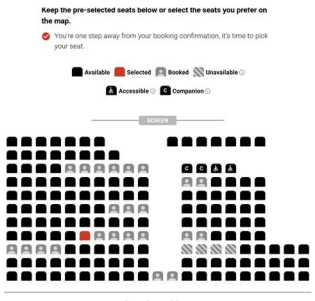
Having selected a date and time in the Everyman website, a user is sent to the seat selection page pictured to the right. I will copy from this the presence of a colour coded seat map and the key. With the Everyman website, there is a lot of white space to the right of this. I will likely replace this in my own section with a way of selecting which kinds of people will be attending (as the brief dictates that children and the elderly have tickets at half price).



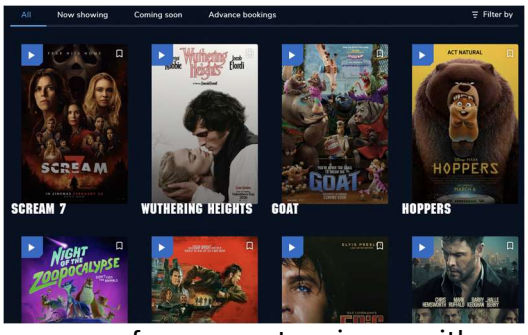
Cineworld has a very similar way of displaying what films are on (pictured right under Everyman). Again, I will draw inspiration from this with the vertical lines. One thing that Cineworld lacks compared to the Everyman is the description of each film on this page with a user having to go into the furtehr details of the film to see. I personally prefer Everyman's approach and will be incorporating it into my design. This difference could be as a reuslt of Everyman's propensity to show more indie and unknown films with the cinema itself being the event rather than the film. This means that they have to give a little more information about the films as people may not have heard of them whereas people would go to Cineworld with the intention of watching a specific film that they have already picked out.



Cineworld makes a user select the number of seats that they want before going to the page for selecting seast (prictured right). I believe this is clunky and convelouted in a way that I do not want to incorporate into my own design. It also mean that in selection a user has to go through a complicated process of queueing up seats to unselect a seat they don't want if they don't want to go with the seats that were automatically assigned to them.



My third case study is the Odeon. It's film selection page is pictured right. While I like the overall layout of this design as it is incredibly pleasing to the eye, it does have some issues that mean I don't think I will be incorporating it into my own designs. For starters, it took an incredibly long time to load with all of the images. While it is entirely possible that this was because of a momentary issue with my internet, it cannot be discounted. Furthermore, I do not like the lack of the descriptions.



I feel that in a school environment detailing the contents of lectures issued by guest speakers and things that people won't have heard of, this could be counterproductive. This would be because people may not recognise the faces of the speakers or performers and as a result may not be inclined to click on the thumbnails for further information.

The seat booking page for the Odeon is now pictured right. I continue to like their dark theme but don't think it would be appropriate for an academic website that would probably try to take itself a little more seriously. One thing that stands out from this platform as opposed to theothers is the presence of classed seating. This is a great idea but, again, I don't think it would be approprate for a college website. One thing that is good is that it automatically selects what it things would be the "best seats" when you selct a class. This could be incorporated into a stretch target quite easily and I belive it could be most helpful.

Ultimately, all platforms have strong foundations and have been built on years of experience that I can draw inspiration from. I personally prefer the way that the Everyman uses and will be using manyof their design principals in my own work. That being said, the differences are very minor. This will mean that users will be used to a certain way of doing things. If I stick to this template, I be able to leave minimal instructions to users and I will be able to inherrit their experience with other similar platforms.

Stakeholders

Performers:

Performers are the people on the stage that will be performing the pieces on the stage. They could be actors, dancers, comedians, guest speakers or any number of people with different needs. While they will not interact directly with the booking system, they will be impacted by the functionalities it has.

It is possible that they will need special accommodations made with seats being closed off. That could be for their families, to give them adequate space to move, or because they may have special movement needs like being in a wheelchair.

As such, it will be important for seats to be blocked off for these purposes by admin staff so that the general public cannot buy specific seats.

Admin Staff:

Admin staff are the people who will be interacting with the management side of the program. They will have to be able to:

- Add new events,
- Block of seats for specific events,
- Add times for events,
- View and edit user types (standard, admin, special guest),
- Search for specific users,
- View which users are attending each showing,
- See how many seats are left at each showing,
- See how much revenue each showing it bringing in,

It could also be helpful for admin staff to be able to search for specific users within showings for the validation of tickets as well as book people in on their own system.

Admin staff may want to check tickets at the door by viewing the printout copy and comparing the name to the people attending. While the names will be sorted alphabetically by surname, it could be helpful to have search functionality. That being said, it is not necessary as they will be able to go through the names by alphabetical order. Furthermore, many events do not check tickets against their platform and instead opt to take tickets as what they appear to be. As such, this will be added to the stretch targets.

It could also be helpful to admin staff to be able to book tickets themselves for other people. This would be similar to staff at cinemas being able to buy you a ticket.

This would not be possible with the current system. The main demographic that would find this helpful would be the elderly and those without technology access. The use of face-to-face systems like this is falling in the current climate so the use of something like this may not be what it would have been ten years ago. Furthermore, the elderly can be seen (from a business point of view) as a less valuable customer on account of their reduced ticket price. As such, it would not be economical to go to such great lengths to accommodate them. That being said, it would help to reduce the digital divide and build a more accommodating society. Ultimately, it would be beneficial to have the ability for admin staff to be able to book seats even if the functionality may not be used. As such, it will be added as a stretch target.

Visitors:

Visitors make up the majority of the user base for this program and their side of the systems will need the most attention as a result. Visitors should be assumed to be new to the platform and will therefore need more instructions and directions for what to do compared to admin staff that would have experience and training in the software.

They will have to be able to:

- Find events and showings that they want to watch,
- Select seats,
- Book tickets,
- Print tickets,
- View previous bookings,

While this is the most critical part of the system, it is also the most intuitive owing to the fact that everyone (myself included) has had extensive experience with them. This will mean that I can draw inspiration from other similar systems (see above) and draw on users' knowledge on how to use them to reduce the need for instruction.

Disabled Visitors:

While the disabled community makes up a small percentage of the British population (approximately 4.9% of the British population have blue badges), Collyer's is a large institution and it would be reasonable to assume that it has a number of students and carers with disabilities that would require special allowances. There is little that the system itself would need to do to accommodate for these people as the bulk of that responsibility falls on the event organisers.

Most modern browsers natively support accessibility features such as high contrast colours and text to speech with minimal input from developers which removes most of the workload from my end. Even then, I will have to ensure that elements are ordered and named sensibly to ensure that these features still work.

It could also be helpful to include a phone number on the receipt page for contact if a member of the party has a disability that may require special allowances to be made. This will ensure that they are adequately provided for and offloads liability from the software house to the college.

Limitations of the Current System

The brief did not mention any previous system. As such, I am led to believe it was simple pen and paper over the phone or in person.

This would have been cumbersome and prone to error. It would require a member of staff to constantly be present at the phone to take bookings. Furthermore, their handwriting would have to be legible enough for people to check if people have bought tickets. It may also not be organised in a sensible way leading to difficulty searching for names on entrance. Finally, it would be prone to error as humans make mistakes. This could be in double booking seats, putting people down for the wrong event or time, or miscalculating how many seats are still available.

Overall, this assumption of the current system demonstrates a great need for innovation to bring it forward to the 21st century and ensure that operations run smoothly.

Aims & Objectives

Threshold:

- Users can view events, times and book specific seats for them;
- Users can download a PDF version of the ticket;
- Admin can view all users and upgrade them to admin or special guest status;
- Input types:
    - Text entry,
    - Buttons,
    - Multi select buttons,
    - Number entries,
    - Date selects,
- The system must allow tickets to be booked. For each ticket sold or allocated, the system must record the name of the customer, type of customer, phone number, performance date, seat booked, price paid;
- The system must prevent a seat, for a particular performance, being sold or allocated more than once;
- The system must allow seats to be blocked so that they cannot be purchased or otherwise allocated;
- The system must allow a staff member to search for a customer and display which ticket(s) they have booked for which performance(s);
- The system must display a list of ticket holders (seat number, name, phone number), sorted by surname, for a selected performance;
- The system must save all relevant data to an external file (so that the system can be closed down at the end of each day and used again the following day);
- The system must produce management information for each performance:
    - Display how many tickets have been sold (or otherwise allocated),
    - Display how many tickets remain available for sale,
    - Display the total revenue for each performance,

Target:

- Show dates on overview of all events to book;
- Allow performances to be edited after adding to the database;
- Allow visitor users to edit their profile information (including password);
- Make use of the logging module;
- Program some automated tests that can be used to check code;
- Use a normalised database in Third Normal Form;

- Exceed the target of only having three nights to book from by allowing admin to add new performances and showings;
- Display a seating layout for each performance that colour codes the seats that have been sold (or allocated) and those that can still be sold (or allocated);
- From the seating layout, allow the staff member to click on one or more seats to allow those seats to be sold (or allocated);
- Prevent single seats being left in the middle of rows;

Stretch:

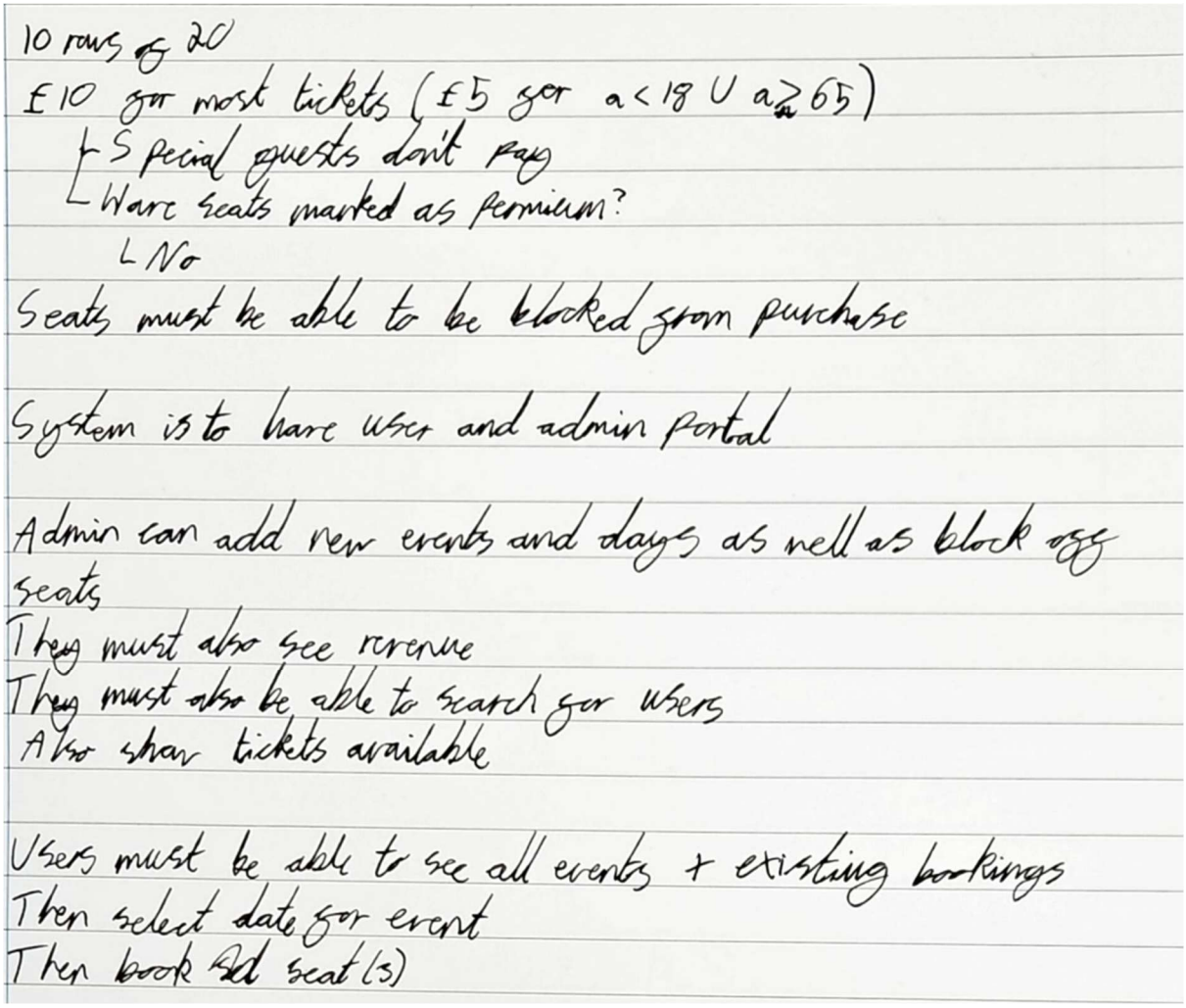
- Have separate timeslots for the day;
- Search users coming to specific showings;
- Admin can book seats;
- Admin can print tickets;
- Automatically select “best seats”;
- Use cookies for login details;
- Make use of a custom API to handle user requests;
- Implement a smart pricing algorithm that takes into account the number of tickets sold and the time remaining until the performance;

Full code and README can be viewed/downloaded from my GitHub.
Username: GeorgeCooke8809

Design

Abstraction

I like to complete the process of abstraction on paper by hand, my paper abstraction of the brief is attached below.



Overview

I will be making use of Flask for my application. This is both because it is more realistic to what a client would want with it enabling users to book online from anywhere but also because I think it is the best option for me. I am inexperienced with PyQt and would rather keep my distance from it. I would be open to using tkinter or custom tkinter but I believe that there would be great performance issues on the booking page as it has to render 200 different seats. As a result, I believe that a web page is the most appropriate choice.

My tool will have two separate portals. One for users and one for admins. Admins will have to be able to:

- Add new events,
- Block of seats for specific events,
- Add times for events,
- View and edit user types (standard, admin, special guest),
- Search for specific users,
- View which users are attending each showing,
- See how many seats are left at each showing,
- See how much revenue each showing it bringing in,

Users will have to be able to:

- Find events and showings that they want to watch,
- Select seats,
- Book tickets,
- Print tickets,
- View previous bookings,

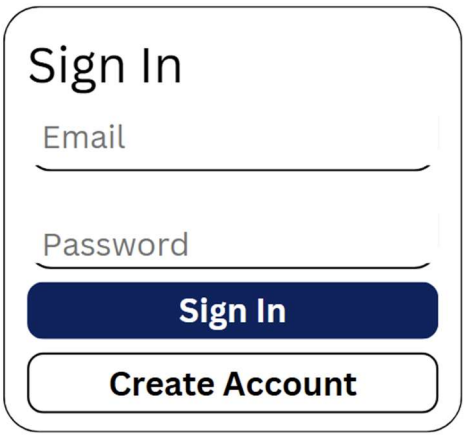
I will also have to make sure there is a GUI for the selection of specific seats. This will ensure that users know where they will be sitting and can coordinate with other parties. One way I could design this within my code is with a for loop. This would save me the effort of writing out each different item by hand and would make my code more efficient. This could be done in the JavaScript or python embedded in the html.

UI

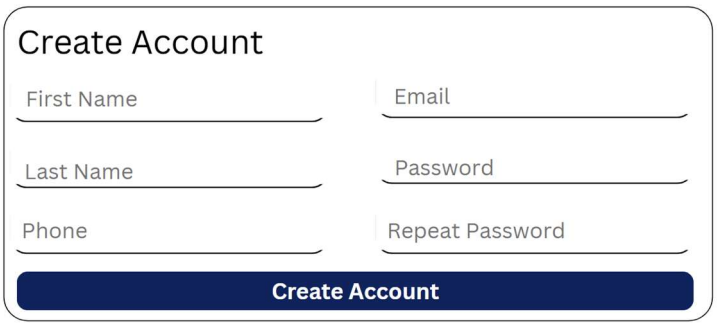
For my colour scheme I will take inspiration from the Collyer's website and take the colour of their banner for my own. This will ensure that I incorporate the Collyer's colour scheme. This will then leave my colour scheme as is shown below:

I elected to make my UI designs in Canva prior to designing any databases or programming any functionality. This is because it is the thing that I am most used to interacting with and I will thus be better equipped to visualise what a user would expect through this medium. I will then go on to use this as a tool to help me make flow charts and diagrams explaining all of the different subroutines I will need to make.

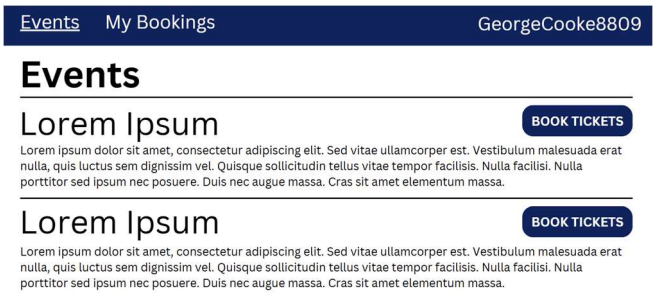
My first designs are for the login and create account page. This will be the index page that a user is sent to first (pictured right). My designs for the car park website were quite bubbly and over simplistic. I would like to maintain the modern aesthetic in this project without going to the bubbly look that makes it look excessively simple. I hope that these designs will help me adhere to this design ideology.



Note the use of "repeat password" in the create account section. This will form part of my validation so as to ensure users have typed the passwordd they intended to type. I will likely try to find a way and have the dots obscurring passwords in the sign in page. This is largely superficial as data transmitted will not be encrypted and my database will not be hashed but I hope it at least resembles security.



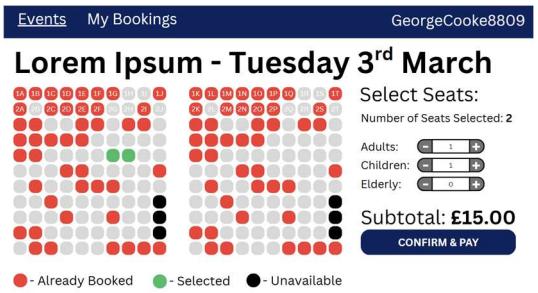
My next page of designs is for my main user dashboard page. This is where a visitor will be able to see all of the events that are being shown and a brief description of them. The book tickets button will take them to the next design.



The navigation bar at the top of the pictured page is the same across all visitor pages. The events button will take them to this page from any page they are at, the "my bookings" button will take them to a list of all their bookings (shown later) and their username in the top right will return them to the index page for log in if pressed.

Having pressed the book tickets button on the previous page, they will be taken to the page shown to the right. This will contain a dynamic list of all the events that are not fully booked and are in the future (or present day). The dates on which events will take place can be set by the administrator.

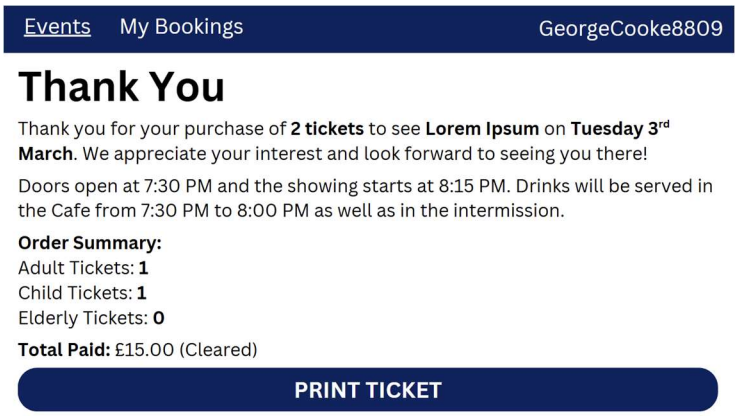
Following on from the page above, the user will be taken to the page shown to the right. This will show a seating map of all the available seatss which will be colour coded. On the right hand side of the screen, they will be able to select the number of people of each category that are in the party. The price will use JavaScript to refrence a custom Flask API that will return the price with the provided criteria (it will also take user ID so as to check if the user is a special guest); this will be refreshed whenever the types of people in the party is refreshed. If they attempt to confim and pay with the number of seats selected and the people in the party being in disagreement, it will flash a message to the bottom right of the screen and not allow them to proceed until they are in agreement.



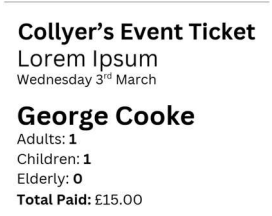
In this graphic, on the top two rows of seast are numbered. In the final product, all seast will be numbered.

I may add a graphic showing where the stage is located to this page if space is available.

The page after this will be the recipt page thanking the user for their booking. This will have a brief overview of what they bought and a thanking message along with arbitary information about when the doors will open (this information will be the same for all events). It will also giove users an opportunity to print their ticket then which will download a PDF copy of the following page.

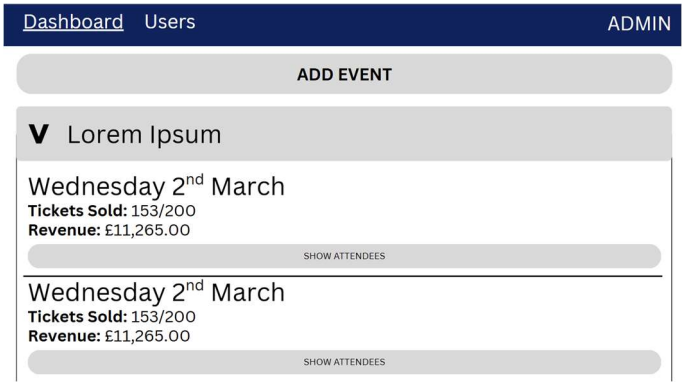


The image shown to the right is a copy of what a ticket/reciept will look like. It is not exceptionally complex but nor does it have to be. It is not designed to be scanned by administrators but they will instead have to either take it at its face value or look the user up in the relevent event bookings list.



Having made a booking, a user will then be able to go to the “My Bookings” page to view all of their active bookings. This will show a list of all of their present and future bookings with the title of the performance, the date of the event they are booked on to and an option to print the reciept/ticket again should they have missed the opportunity the previous time around.

Moving on from the visitor portal, we move into the admin protal. The image shown to the right shows the main admin dashboard. This is where they will be able to:

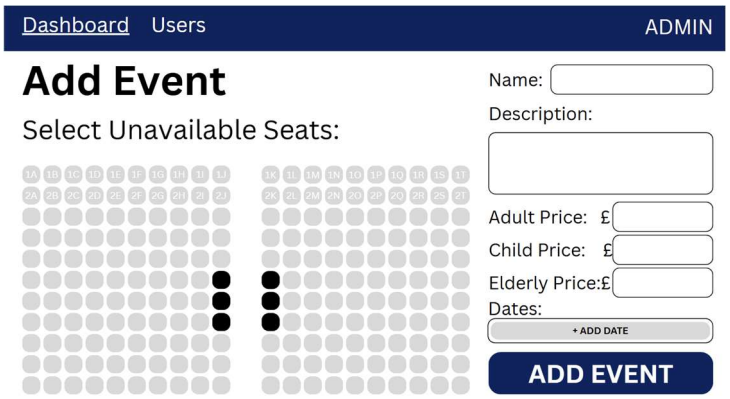


- View all performances,
- View all showings of performances,
- Add new evetns,
- Add new showings,
- View who is attending each showing,

To prevent clutter, performances will initially be shown collapsed. This will hide any information about space, tickets sold, and revenue. Upon expanding a performance, the admin user will be met with an overview of each showing. Still collapsed will be the information about specific visitors for each showing. This can be expanded by pressing the “Show Atendees” button. When expanded, attendees for the event will be shown ordered by surname in alphabetical order.

Similarly to the user portal, the admin portal has the same naviagtion bar at the top of all windows. This shows the main tabs (“Dashboard”, and “Users”) as well as a button showing they are logged in as an admin which will log them out if pressed.

The image shown to the right it the page that an admin user will use to add a new event. I will likely change the scaling of the panel to the right here so as to give more space to the dates and description fields.



The seat map to the left will initially show all seast as being available (grey) but a admin user will be able to click on each seat they want to exclude from booking. This will make it unavailable for booking. This could be done to reserve it for members of the performance or to enhance accessability of the stage. Similar to the bookings page, all seats will be numbered as they are on the top two rows.

The admin user can also set the name, description, and prices here. I will likely autofill the prices to the deafult £10 and £5 values that I was provided in the original brief. Adding a date will be the equivalent of adding a showing in the dashboard page and when the button is pressed it will open a date select page. The list of dates will then be added to the box. On overflow, a scroll bar will appear so as not to change the formatting of the page.

On this page, all fields will be mandatory. If a field is left balnk, an error message will be flashed and the admin user will not be allowed to proceed.

One limitation of this system is that is assumes that the same seats will be marked as unavailable for all showings of a performance. It also does not allow different prices for different showings.

The image shown to the right is the last of the admin pages and will show a list of all users sorted in alphabetical order by surname. In this page, an admin user wil lhave the ability to view user names and phone numbers as well as mark useres as admin or special guests. Admin users will be sent to the admin portal when loggin in. Special guests are the people mentioned in the brief that get free tickets. Marking a user as one of these will ensure they receive free tickets. If a user is already marked as one of these, the relevent button will be changed to have the prefix “un-” e.g.: “UN-ADMIN”. A admin user also has the ability to delete a user from this page.

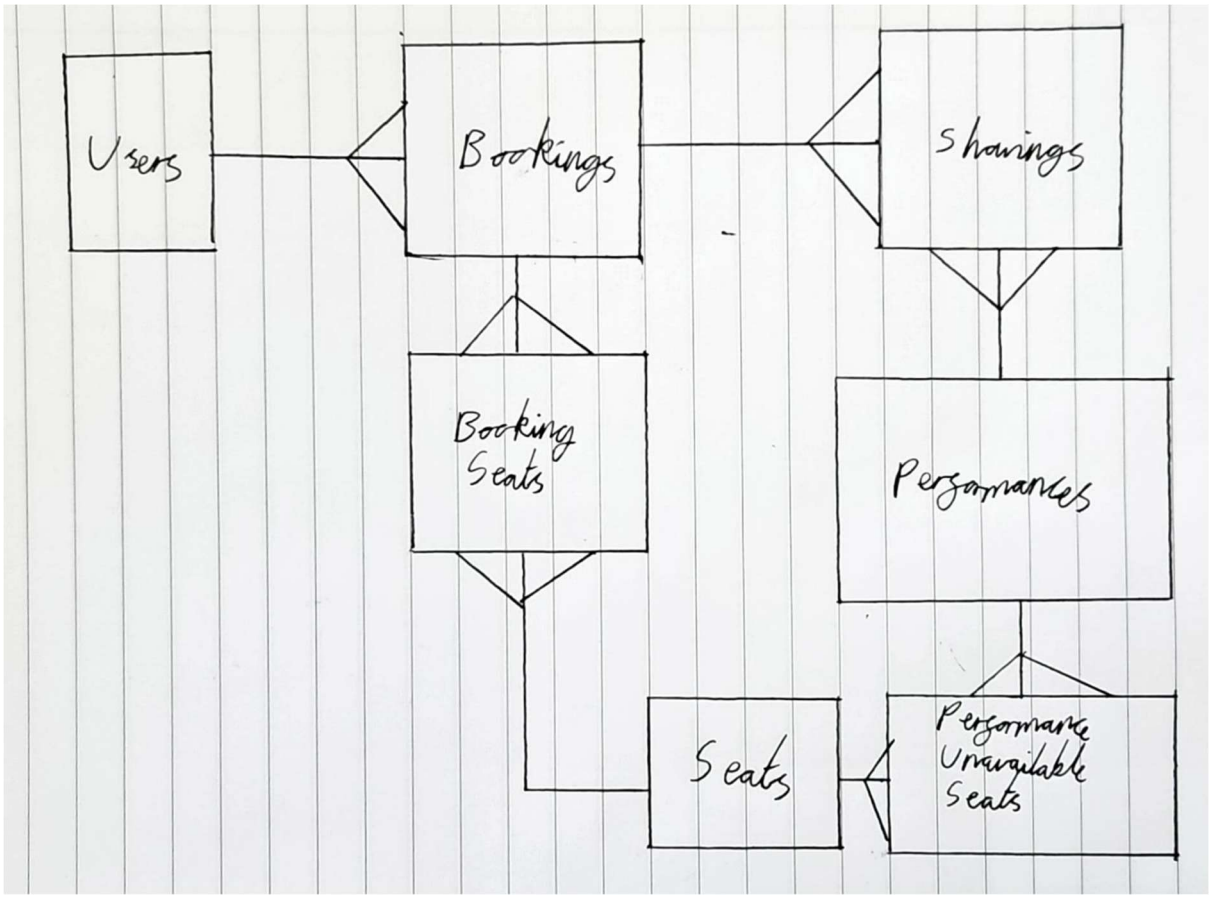
Systems

Database

Tables:

- Users(UserID, firstName, lastName, email, password, phone);
- Bookings(BookingID, ShowingID*, UserID*);
- BookingSeats(BookingSeatID*, BookingID*, SeatID*, type);
- Performances(PerformanceID, title, description, childPrice, adultPrice, elderlyPrice);
- PerformanceUnavailableSeats(PerformanceUnavailableSeatsID*, PerformanceID*, SeatID*);
- Seats(SeatID);
- Showings(ShowingID, PerformanceID*, Date);

ERD:



Data Dictionary:

Users

Field Name	Field Data Type	Field Purpose	Field Validation	Example Data
UserID	INT	Unique identifier for every user	Automatically generated	1 2 3
firstName	VARCHAR(30)	First name of user	Length check and suitable chars (alphabet, -, ‘)	George Jamie Olly
lastname	VARCHAR(30)	Surname of user	Length check and suitable chars (alphabet, -, ‘)	Cooke Allsop Kitson
email	VARCHAR(100)	Email of user for contact	Length check, has one at symbol followed by one full stop (at some point), check is not already in database	Georgecooke8809@gmail.com 25cookeg899@collyers.ac.uk 20gcooke@wardenparkradio.net
password	VARCHAR(50)	Password for logging in	Checked is the same as re-entry	Password123 Kvjwhvhhlau8y2lhg avjbiluahgl
phone	VARCHAR(16)	Phone number for contact about bookings	Only contains numbers and spaces or hyphens, length check	07802 447389 07801 749295 07744 456892
userType	VARCHAR(	The type of user (admin, special guest, visitor)	Is passed in by buttons so none needed	VISITOR ADMIN SPECIAL

Bookings

Field Name	Field Data Type	Field Purpose	Field Validation	Example Data

BookingID	INT	Unique identifier for every booking	Automatically generated	1 2 3
ShowingID	INT	Unique identifier for every showing to show which show the booking is for	Automatically generated	1 2 3
UserID	INT	Unique identifier for every user showing who booked the booking	Automatically generated	1 2 3

Booking Seats

Field Name	**Field Data Type**	**Field Purpose**	**Field Validation**	**Example Data**
BookingSeatID	INT	Unique identifier for every booking seat	Automatically generated	1 2 3
BookingID	INT	Unique identifier for every booking seat	Automatically generated	1 2 3
SeatID	VARCHAR(3)	Unique identifier for every booking seat	Automatically generated	1A 2B 5C
bookingType	VARCHAR(7)	The type of visitor using the seat	One of the three valid options	CHILD ADULT ELDERLY

Performances

Field Name	**Field Data Type**	**Field Purpose**	**Field Validation**	**Example Data**
PerformanceID	INT	Unique identifier for every performance	Automatically generated	1 2 3
title	VARCHAR(100)	The title of the performance	Length check	Hamilton

				Star Wars: Episode IV - A New Hope Star Wars: Episode III - Revenge of The Sith
performanceDescription	VARCHAR(MAX)	The description of the performance	None	...
childPrice	DECIMAL(2)	The price for a child ticket	Two decimal places, >= 0	5.00 7.50 10.12
adultPrice	DECIMAL(2)	The price for an adult ticket	Two decimal places, >= 0	5.00 7.50 10.12
elderlyPrice	DECIMAL(2)	The price for an elderly ticket	Two decimal places, >= 0	5.00 7.50 10.12

Performance Unavailable Seats

Field Name	**Field Data Type**	**Field Purpose**	**Field Validation**	**Example Data**
PerformanceUnavailableSeatsID	INT	Unique identifier for every performance	Automatically generated	1 2 3
PerformanceID	INT	Unique identifier for every performance	Automatically generated	1 2 3
SeatID	VARCHAR(3)	Unique identifier for every unavailable seat	Automatically generated	1A 2B 5C

Seats

Field Name	**Field Data Type**	**Field Purpose**	**Field Validation**	**Example Data**
SeatID	VARCHAR(3)	Unique identifier for every seat	Automatically generated	1A 2B 5C

Showings

Field Name	Field Data Type	Field Purpose	Field Validation	Example Data
ShowingID	INT	Unique identifier for every showing	Automatically generated	1 2 3
PerformanceID	INT	Unique identifier for every performance to show the performance the showing is for	Automatically generated	1 2 3
showingDate	DATE	The date the showing is on	Passed in by a date select that provides a consistent format, range check (on or after current date when added)	03/03/2026 01/04/2026 07/05/2026

Decomposition & Algorithms

1.0.0 Log In

1.1.0 Log In
- 1.1.1 Check log in details,
- 1.1.2 IF LOG IN DETAILS CORRECT: send user to relevant portal,
- 1.1.3 IF LOG IN DETAILS INCORRECT: flash error message,

1.2.0 Create Account
- 1.2.1 Take in user details,
- 1.2.2 Validate user details,
    1) First name and last name are valid length and contains only letters,
    2) Email is valid length, has @ symbol followed by "." (at some point), and not already in database,
    3) Phone has only numbers,
    4) Passwords are the same,
- 1.2.3 IF CREATE USER DETAILS VALID: pass into database,

1.2.4 IF CREATE USER DETAILS INVALID: flash error message

2.0.0 User Portal

2.1.0 View available performances,
- 2.1.1 Get list of all performances with space available and events after or on current date,
- 2.1.2 Show list of all performances,
- 2.1.3 Select date for event,
  - 2.1.3.1 Get all dates for performance where date is after or on current and not full,
  - 2.1.3.2 Show all valid dates for performance,
  - 2.1.3.3 Book seats,
    - 2.1.3.3.1 Select seats,
    - 2.1.3.3.2 Select party composition
    - 2.1.3.3.3 Get price,
    - 2.1.3.3.4 Check party size agrees with seat selection
    - 2.1.3.3.5 IF PARTY SIZE DISAGREES: flash error message and prevent proceed
    - 2.1.3.3.6 IF PARTY SIZE AGREES: Submit to database

2.2.0 View booked performances,
- 2.2.1 Get all booked performances after and including current date,
- 2.2.2 Show all valid bookings,

2.3.0 Print ticket for performance – inside booked performances page,
- 2.3.1 Generate PDF,
- 2.3.2 Download PDF,

2.4.0 Log out,
- 2.4.1 Redirect to index page,

3.0.0 Admin Portal

3.1.0 Dashboard page
- 3.1.1 Add event,
  - 3.1.1.1 Get data and add to custom internal API
    - 3.1.1.1.1 Select unavailable seats,
    - 3.1.1.1.2 Set title,
    - 3.1.1.1.3 Set description,
    - 3.1.1.1.4 Set prices,
      - (i) Child,
      - (ii) Adult,
      - (iii) Elderly,

3.1.1.1.5 Set dates,
3.1.1.2 Add event,
3.1.1.2.1 Create new event,
3.1.1.2.2 Add showings to event,
3.1.2 View event,
3.1.2.1 Display showings,
3.1.2.1.1 Get showings (ordered by ascending date)
3.1.2.1.2 Display basic information,
(i) Seats available,
(ii) Revenue,
(iii) Date,
3.1.2.1.3 Display attendees,
3.1.2.2 Add new showing,
3.1.2.2.1 Get date,
3.1.2.2.2 Add showing,
3.2.0 Users page
3.2.1 Get all users
3.2.1.1 Get a list of all users,
3.2.1.2 Return and display the list of all users
3.2.1.2.1 Get list of users,
3.2.1.2.2 Determine if admin or guest already
3.2.1.2.3 Display with relevant buttons
3.2.1.3 Toggle admin of user,
3.2.1.3.1 Set type to user or special as required,
3.2.1.4 Toggle special of user,
3.2.1.4.1 Set type to user or special as required,
3.2.1.5 Delete user,
3.2.1.5.1 Delete user,
3.3.0 Log out
3.3.1 Redirect to index page

Inputs & Outputs:

Input	Location	Processing	Output
Email text entry	Sign in page (1st row)	-	-
Password text entry	Sign in page (2nd row)	-	-
Sign in button	Sign in page (3rd row)	Validates log in information and redirects to relevant dashboard	Redirects to relevant dashboard
Create account button	Sign in page (4th row)	Redirect to create new account page	Redirect to create new account page
First name text entry	Create account page (1st row, 1st column)	-	-
Last name text entry	Create account page (2nd row, 1st column)	-	-
Phone text entry	Create account page (3rd row, 1st column)	-	-
Email text entry	Create account page (1st row, 2nd column)	-	-
Password text entry	Create account page (2nd row, 2nd column)	-	-
Repeat password text entry	Create account page (3rd row, 2nd column)	-	-
Create account button	Create account page (bottom row)	Checks email is not in database, checks passwords are the same, passes data into database, redirects to sign in page	Redirects to sign in page
Events button	Top left of every visitor page	Redirects to events page	Redirects to events page
My bookings button	Top left (ish) of every visitor page	Redirects to my bookings page	Redirects to my bookings page

Input	Location	Processing	Output
Log out button (displayed as full name)	Top right of every visitor page	Redirects to my bookings page	Redirects to my bookings page
Book tickets button	To the left of every performance in the main visitors events page	Redirects to date (showing) select for relevant performance	Redirects to date (showing) select for relevant performance
Date select button(s)	The buttons in the main body of the performance date selection page	Redirects to relevant booking page	Redirects to relevant booking page
Seat select buttons	The left column of the booking page	Toggles the colour of the button and toggle the relevant seat from a list of seats to be added to the booking	Toggles the colour of the seat
Number of adults seats number select	Right hand column, 1st row of booking page	-	-
Number of children seats number select	Right hand column, 2nd row of booking page	-	-
Number of elderly seats number select	Right hand column, 3rd row of booking page	-	-
Confirm & Pay button	Bottom of right column in booking page	Checks number of seats selected and seat type numbers agree, push to database and redirect to thank you page	Redirect to thank you page
Print ticket button	Thank you page	Create PDF ticket for relevant booking and send to user for download	PDF ticket downloaded
Print ticket button	My bookings page	Create PDF ticket for relevant booking and send to user for download	PDF ticket downloaded
Dashboard button	Top left of every admin page	Redirect to admin dashboard page	Redirect to admin dashboard page
Users button	Top left (ish) of every admin page	Redirect to admin users page	Redirect to admin users page
Logout button (displayed as admin)	Top right of every admin page	Redirect to sign in page	Redirect to sign in page

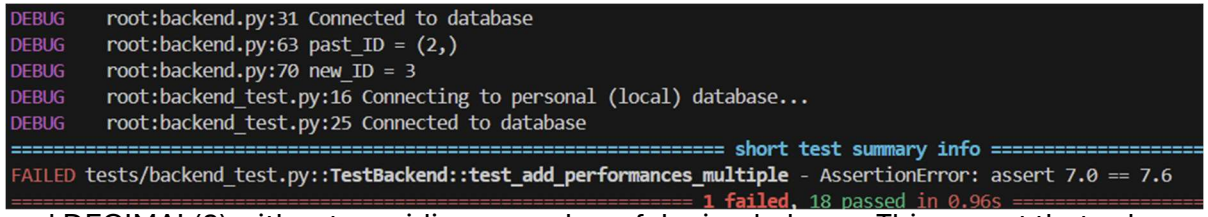
Input	Location	Processing	Output
Expand event button	The arrow next to every event entry in the admin dashboard page	Expand selection	Expand selection
Show attendees	Under each showing of an event when the event is expanded	Expand selection	Expand to show people booked on to showing
Add showing date select	At the bottom of the expanded performance information	Adds new date to performance	Refreshes page and shows new showing
Seat buttons	Add event page for admin	Toggle colour of seat and toggle from array for later processing	Toggle colour of seat
Name text entry	1st row of right column in add event page	-	-
Description text entry	2nd row of right column in add event page	-	-
Adult price number entry (decimal)	3rd row of right column in add event page	-	-
Child price number entry (decimal)	4th row of right column in add event page	-	-
Elderly price number entry (decimal)	5th row of right column in add event page	-	-
Add date date select	6th row of right column in add event page	Adds date to list of dates and shows it in the scrollable frame	Shows the date in the list
Date button	In the 6th row of the right column in add event page	Clears the date from the list and hides it	Removes date from the list
Add event button	Bottom of the right column in add event page	Validates all the date entered and adds to database then redirects to admin dashboard	Redirects to admin dashboard
Admin button	Users page	Toggles if a user is an admin user	Refreshes the page

Input	Location	Processing	Output
Special button	Users page	Toggles if a user is a special guest and refreshes the page	Refreshes the page
Delete button	Users page	Deletes a user and refreshes the page	Refreshes the page

Testing

Development Testing:

Error #1:



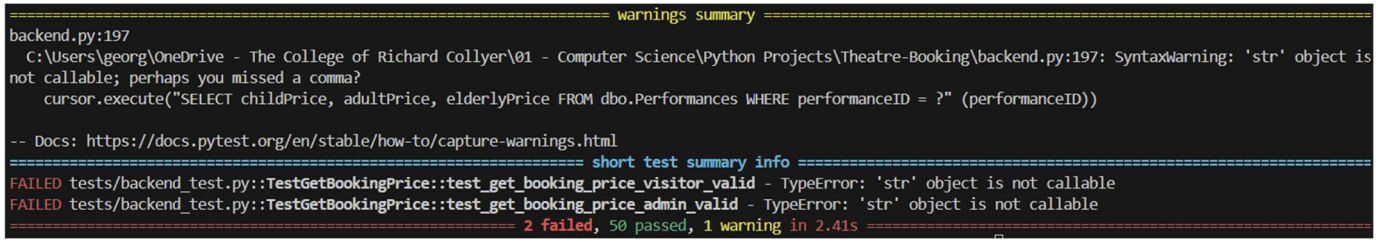
The error pictured to the right occurred when trying to add functionality to custom prices for performances. My error was in using the incorrect data type in setting up my database. I used DECIMAL(2) without providing a number of decimal places. This meant that values were automatically rounded to the nearest whole number. A similar issue occurred when trying to insert values for price with any more than 2 digits before the decimal point. Corrected by changing SQL table creation command to DECIMAL(7, 2)

Error #2:

```
child_split = str(child_price).split(".")
adult_split = str(adult_price).split(".")
elderly_split = str(adult_price).split(".")
```

This was a simple error that came from me copy and pasting this code so as to save myself time. You can see in the screenshot that I forgot to change the str(adult_price) in the third line to elderly_price. This meant that it was not correctly validating the number of decimal places in the number which led to errors. Fortunately, this was caught during my unit testing.

Error #3:



This error was another simple one. You can see in the warnings summary as well as the code screenshot below that I forgot to

add the comma in my SQL statement where I insert the variables. It read (“SELECT childPrice, adultPrice, elderlyPrice FROM dbo.Performances WHERE performanceID = ?” (performanceID)) where it should have read (“SELECT childPrice, adultPrice, elderlyPrice FROM dbo.Performances WHERE performanceID = ?”, (performanceID)).

```
logging.debug("Getting prices...")
cursor.execute("SELECT childPrice, adultPrice, elderlyPrice FROM dbo.Performances WHERE performanceID = ?" (performanceID))
prices = cursor.fetchone()
```

Error #4:

This is another simple error that stems from me using the incorrect data type in my declaration of tables. In my data dictionary, I declared seatID to be VARCHAR(3) but I unfortunately neglected to do this when actually making the table and defaulted to INT.

```
=========================== short test summary info ============================
FAILED tests/backend_test.py::TestMarkSeatUnavailable::test_mark_seat_unavailable_valid - pyodbc.DataError: ('22018', "[22018] [Microsoft][ODBC Driver 18 for SQ
L Server][SQL Server]Conversion failed when converting the nvarchar value '1B' to dat...
FAILED tests/backend_test.py::TestMarkSeatUnavailable::test_mark_seat_unavailable_multiple_valid - pyodbc.DataError: ('22018', "[22018] [Microsoft][ODBC Driver
18 for SQL Server][SQL Server]Conversion failed when converting the nvarchar value '1A' to dat...
========================== 2 failed, 58 passed in 2.85s ========================
```

Final Testing:

Test No	Test Purpose	Test Steps	Expected Output	Real Output	Pass/Fail	Comment
1	Check redirect of create account link from main login page	Press create account button from main login page	Redirect to create account page		P	
2	Check create account works with valid data	Enter following data: First Name = George, Last Name = Cooke, Phone = 07802 447089, password = 25cookeg899@collyers.ac.uk, password = Password, confirm password = Password	User is added to the database with all given data and is redirected to login screen		P	
3	Check create account works with erroneous phone number (not a number)	Enter following data: First Name = George, Last Name = Cooke, Phone = a, password = 25cookeg899@collyers.ac.uk, password = Password, confirm password = Password	Flash message appears, user is not redirected and no data is added to database		P	
4	Check create account works with fields not filled in	Fill all data in with correct data (see test 2) then repeatedly test with one bit of data missing until all fields have been tested.	Flash message appears, user is not redirected and no data is added to database		P	
5	Check create account works with invalid email address	Enter following data: First Name = George, Last Name = Cooke, Phone = 07802 447089, password = 25cookeg899@collyers, password = Password, confirm password = Password	Message box appears under email field saying incorrect format when trying to submit and prevents submission.		P	

6	Check create account works with non-matching passwords.	Enter following data: First Name = George, Last Name = Cooke, Phone = 07802 447089, password = 25cookeg899@collyers.ac.uk, password = Password, confirm password = Password123	Flash message appears, user is not redirected and no data is added to database		P	
7	Check login works with valid login information for visitor.	Login with valid login data from the database	User is redirected to the visitor home page with correct UID.		P	
8	Check login works with valid login information for special guest.	Login with valid login data from the database	User is redirected to the visitor home page with correct UID.		P	
9	Check login works with default admin information.	Login with default admin login credentials.	User is redirected to the admin dashboard.		P	
10	Check login works with database admin credentials.	Login with valid admin login from the database.	User is redirected to the admin dashboard.		P	
11	Check login works with fields empty.	Leave fields empty and try to login.	Flash message appears and user is not redirected.		P	
12	Check login works with incorrect login details for email in database.	Put valid email in but incorrect password.	Flash message appears and user is not redirected.		P	
13	Check login works with details that are not in the database anywhere.	Put nonsensical fields into the login page.	Flash message appears and user is not redirected.		P	
14	Check login works with password that is for another email.	Put email and password that are in the database but for different accounts.	Flash message appears and user is not redirected.		P	

15	Check pressing ADMIN in top right of admin dashboard logs out.	Click name in top right of admin dashboard.	Redirected to login page.		P	
16	Check clicking on users in admin nav bar redirects to admin users page.	Click on users in admin nav bar from dashboard.	Redirect to admin users page.		P	
17	Check pressing name in top right of admin users logs out.	Click name in top right of admin users.	Redirected to login page.		P	
18	Check clicking on dashboard in admin users page redirects to dashboard.	Click on dashboard in nav bar in admin users page.	Redirect to admin dashboard page.		P	
19	Check add performance works to redirect from admin dashboard.	Press add performance button in admin dashboard.	Redirect to add performance page.		P	
20	Check dashboard button redirect from add performance page.	Press dashboard in the nav bar of add performance page.	Redirect to admin dashboard page.		P	
21	Check users button redirect from admin add performance page works	Press users in nav bar of add performances page	Redirect to admin users page.		P	
22	Check admin logout button works in admin add performance page works	Press ADMIN in nav bar of add performance page.	Redirect to login page.		P	
23	Check admin add performance works with valid data (w/ description, no unavailable seats)	Add valid data to all fields in admin add performance	Performance is added to database and user is redirected to admin dashboard with new performance visible.		P	

24	Check admin add performance works with valid data (no description, no unavailable seats)	Add valid data to all fields in admin add performance except for description	Performance is added to database and user is redirected to admin dashboard with new performance visible.		P	
25	Check admin add performance works with valid data (w/ description, w/ unavailable seats)	Add valid data to all fields in admin add performance and mark seats as unavailable.	Performance is added to database as well as unavailable seats and user is redirected to admin dashboard with new performance visible.		P	
26	Check add performance works with no title.	Do not add title to add performance page.	Flash message appears, data is not added to database and user is not redirected.		P	
27	Check add performance works with no price.	Delete prices from add performance (also add title)	Flash message appears, data it not added to database and user is not redirected.		P	
28	Check performance works when trying to type letter in price field.	Delete prices from add performance and try to type a letter.	Letter cannot be typed.		P	
29	Check performance works with boundary length title.	Type title that is 100 characters long.	Performance is added to database and user is redirected to admin dashboard with new performance visible.		P	
30	Check add performance works with invalid length title.	Type title that is 101 characters long.	Flash message is show, data is not added to database, user is not redirected.		P	

31	Check add showing button works with adding showing from previous date	Set date of performance to date in the past	Flash message appears saying date is in the past and nothing is added to DB	Allows adding performance	F	Add a quick check in backend.py and main app
32	Check add showing works with valid date	Set date of performance to date in future	Showing is added to DB, page refreshes, new showing appears (empty of bookings and with valid revenue (£0))		P	
33	Check add showing works with boundary date	Set date of showing to today	Showing is added to DB, page refreshes, new showing appears (empty of bookings and with valid revenue (£0))		P	
34	Check delete showing button works	Click delete showing button	Showing is deleted with all bookings associated with it from DB and page is refreshed		P	
35	Check delete performance button works	Click delete performance button	Performance is deleted with all showings and bookings associated with it.		P	
36	Check dashboard redirect works in admin view users page	Click dashboard in top left of admin users page	Redirect to admin dashboard page		P	
37	Check admin logout works in admin view users page	Click ADMIN in top right of nav bar in admin users page	Redirect to main login page		P	
38	Check update user to SPECIAL works	Change user type to special	Data is updated in DB and page is refreshed to represent		P	
39	Check update user to VISITOR works	Change user type to visitor	Data is updated in DB and page is refreshed to represent		P	

40	Check update user to ADMIN works	Change user type to admin	Data is updated in DB and page is refreshed to represent		P	
40	Check delete user works	Delete user	Data is updated in DB and page is refreshed to represent		P	
41	Check search works	Search in search bar	Relevant search results appear		P	
42	Check update user to SPECIAL works from search	Search and update a user	Data is updated in DB and page is refreshed to represent		P	
43	Check update user to VISITOR works from search	Search and update a user	Data is updated in DB and page is refreshed to represent		P	
44	Check update user to ADMIN works from search	Search and update a user	Data is updated in DB and page is refreshed to represent		P	
45	Check delete user works from search	Search and delete a user	Data is updated in DB and page is refreshed to represent		P	
46	Check my bookings redirect works from user dashboard	Click my bookings in nav bar	User is redirected to my bookings page		P	
47	Check logout button works in user dashboard	Click user name in top right of nav bar	User is redirected to main login page		P	
48	Check clicking on a performance works	Click on book tickets for a performance	User is redirected to select showing page		P	
49	Check select showing works	Click on a showing	User is redirected to seat selection page		P	
50	Check my bookings works from select showing page	Click on my bookings in nave bar from select showing page	User is redirected to my bookings page		P	

51	Check events works from select showings page	Click on dashboard in nav bar in select showing	User is redirected to dashboard		P	
52	Check logout works from select showing page	Click on user name in top right nav bar of select showing page	User is redirected to main login page		P	
53	Check events works from booking page	Click on events in booking page	Redirected to main dashboard		P	
54	Check my bookings works from booking page	Click on my bookings in nav bar of bookings page	User redirected to my bookings page		P	
55	Check logout works from booking page	Click on username in top right of nav bar in booking page	User is redirected to main login page		P	
56	Check booking works with valid data	Select three seats and 1 from each age bracket	DB is updated and user is redirected to thank you page		P	
57	Check booking works with too few seats	Select three seats from and 1 from two of the age brackets	Flash message appears and progress stopped		P	
58	Check booking works with too few seats	Select 2 seats and 1 from each age bracket	Flash message appears and progress stopped		P	
59	Check events works from thank you page	Click events in nav bar of thank you page	User is redirected to visitor dashboard		P	
60	Check my bookings works from thank you page	Click my bookings in nave bar of thank you page	User is redirected to my bookings page		P	
61	Check log out works from thank you page	Click on user name in top right of thank you page	User is redirected to main login page		P	
62	Check print tickets works from thank you page	Click on print ticket in thank you page	PDF ticket with correct data is created and downloaded		P	
63	Check events works from my bookings page	Click on events in my bookings page	Redirect to user dashboard		P	
64	Check log out works from my bookings page	Click on user name in my bookings page	User is redirected to main login page		P	

65	Check more details works from thank you page	Click on more details for a booking	User is redirected to relevant booking thank you page		P	
66	Check book seats works as a special guest	Book seats as a special account	Price is £0.00		P	

Conclusion & Evaluation

Overview:

Ultimately, I think I did a better job at setting expectations for this project than I did in my previous project (Collyer's Car Park). While I'm sure this was helped by the understanding of HTML, CSS, and JavaScript that I developed previously, I believe that I also demonstrated the setting of more realistic aims and objectives.

Contrary to the previous project, I felt I had ample time to complete the project with it bordering on excess. That may seem ironic as I write this only 3 days before the due date but it has to be taken into account that I took most of the easter holidays off from working on my project.

One of the key take aways from my previous project was an overwhelming lack of validation. One example of this was that I did not check for repeated licence plates. In this more recent project, I made a point of implementing validation at every opportunity. I also implemented flash messages to say why something was wrong and what had to be changed that were not present in my previous project. This was done using a custom Flask API that was referenced by the JavaScript of the website. This was more effective than my previous methods of sending POST requests straight to Flask from the HTML forms as I had greater control over how my data was passed and it meant that I could display flash messages without refreshing the whole page and clearing all form fields in the process. In addition to this, this method allowed me to more clearly organize my code which contributes to its maintainability. This also developed my knowledge of JavaScript.

My use of SQL was similar to in my previous project, and I maintain the idea that there must be a more efficient way of doing it although the solution eludes me. I'm sure that I could have made better use of code comments but I think that my code layout and self-documenting identifiers ought to be enough for an external developer to get the gist of what is being done. I was quite satisfied with my styling from my previous project and believe that I have built on this to make it even better. I remain proud that I did not rely on any third part templates.

I maintain that the use of Flask was the correct choice for this project as it allowed me to have fine and granular control of everything that appeared where I believe PyQt6 would have let me down.

Finally, I am much prouder of myself for making a fully functional program for with both a visitor and admin portal which I failed to do in my previous project.

What I've Learnt:

In the completion of this project, I have developed my understanding of Flask, HTML, CSS, JavaScript and SQL.

While I had used all of these before in both personal projects and the previous project completed for college, I have built on this understanding. I mentioned in the evaluation of my previous project that I maintained a certain aversion to the use of Flask and web apps in general for real world applications as I cited a certain disorganized feeling. I would almost say that this project has won me over with Flask proving once again its flexibility and application. I maintain that there are cases where native apps will thrive over a web app, but I now see the merit of both options fully.

I feel that my use of a custom API demonstrates much of this improved understanding. This acts as a bridge between my website and my backend in a way that is considerably neater than my efforts in previous projects.

I also feel that my use of CSS for a moderna and consistent GUI was improved upon from my previous project. This was no doubt aided by the additional time I had available and my not needing to implement user uploaded images.

I believe that what I have learnt in the completion of this project has made me a more versatile developer that will do me no end not only in my personal projects but also when it comes to making my way into the workforce.

Going Back to the Objectives:

Threshold:

- Users can view events, times and book specific seats for them;
- Admin staff can add events and showings (dates) for events;
- Admin can view all users and upgrade them to admin or special guest status;
- Input types:
    - Text entry,
    - Buttons,
    - Multi select buttons,
    - Number entries,
    - Date selects,
- The system must allow tickets to be booked. For each ticket sold or allocated, the system must record the name of the customer, type of customer, phone number, performance date, seat booked, price paid;
- The system must prevent a seat, for a particular performance, being sold or allocated more than once;
- The system must allow seats to be blocked so that they cannot be purchased or otherwise allocated;
- The system must allow a staff member to search for a customer and display which ticket(s) they have booked for which performance(s);
- The system must display a list of ticket holders (seat number, name, phone number), sorted by surname, for a selected performance;
- The system must save all relevant data to an external file (so that the system can be closed down at the end of each day and used again the following day);
- The system must produce management information for each performance:
    - Display how many tickets have been sold (or otherwise allocated),
    - Display how many tickets remain available for sale,
    - Display the total revenue for each performance,

Target:

- Show dates on overview of all events to book;
- Allow performances to be edited after adding to the database;
- Allow visitor users to edit their profile information (including password);
- Make use of the logging module;
- Program some automated tests that can be used to check code;
- Use a normalised database in Third Normal Form;
- Exceed the target of only having three nights to book from by allowing admin to add new performances and showings;

- Display a seating layout for each performance that colour codes the seats that have been sold (or allocated) and those that can still be sold (or allocated);
- From the seating layout, allow the staff member to click on one or more seats to allow those seats to be sold (or allocated);
- Prevent single seats being left in the middle of rows;
- Users can download a PDF version of the ticket;

Stretch:

- Have separate timeslots for the day;
- Search users coming to specific showings;
- Admin can book seats;
- Admin can print tickets;
- Automatically select "best seats";
- Use cookies for login details;
- Make use of a custom API to handle user requests;
- Implement a smart pricing algorithm that takes into account the number of tickets sold and the time remaining until the performance;

I am satisfied that I met most of the threshold targets but disappointed that I neglected a number of the objectives for no reason other than negligence. I did include information about who was booked on each showing but forgot that it was supposed to also include information about seats. I also neglected that an admin user was supposed to be able to search within this section.

From the target objectives, I made use of the logging module from the outset of my project. This removed the need for countless debugging print statements and meant that debugging information was retained for longer than the character limit in the terminal which allowed for more informed analysis of issues. One thing that I found was that the logging module often wouldn't do what I wanted when using multiple python files. This was counter intuitive and is something that I ought to look into during or before my coursework.

I also made use of pytest for unit tests. This was invaluable when developing my backend but took a back seat when I started work on my frontend. Typically, this wouldn't be much of an issue. However, while working on my frontend, I found that I was missing a number of functions in my backend and one or more of the functions in my backend were not providing data in the best format. This prompted me to make a number of changes to my backend file without taking the time to update my unit tests. In a real-world application, this would be risky at best. Not only did these changes mean

that some functions weren't included in my unit tests but also meant that a number of tests now fail as functions return unexpected results.

Potential Improvements:

If I had my time again, I would ensure that my unit tests were full and complete at every opportunity and would try to add more depth to the capabilities of users as mentioned in the target objectives. This would make my solution more complete.

I would also spend some time going through my backend and grouping functions together by function with comments clearly showing what everything did. This would make my code more understandable and maintainable. I would also spend some time looking for potential efficiency improvements by converting frequently repeated code blocks into functions where possible.

I also know that there is an issue where the program calculates the price paid of a booking as £0.00 if a user is marked as a special guest even if the ticket was bought while the user did not have special privileges. This would have been fixed by adding a field in the bookings table that stored the price paid. At the time, I thought this would have been redundant as price could have just been calculated.

Finally, I would spend a considerable amount of time on the implementation of a responsive UI that can scale for different sized displays. As it stands, my UI looks flawless on my display, but I recognize that on a display with any variation in resolution the UI will look either too crowded or too spread out. This is a large oversight that I had intended to solve with any spare time left at the end of the project.

The above only really gets me to meet a baseline of what is acceptable. If I were to try and make this project something I could truly be proud of, I would push myself further by implementing cookies to store user data. I am sure that Flask has this built in and that I am only being scared of by some perception of difficulty that has no substance.

I also feel that I did a poor job of showing the users that were booked on to a showing. I would want to add the seats that a user is booked for and add a way of searching for specific users and seats in each showing. This would make the program more effective for theatre workers confirming tickets.

In my next project I will endeavour to:

- Implement a responsive GUI,
- Implement cookies,
- Have a better understanding of the logging module,
- Make better use of unit tests,
- Further develop my styling to prevent my UI from looking excessively flat,
- Pre-load the project with sample data for the convenience of the examiner,

Opinions:

While I am satisfied with my project, I would have liked to have gone further and hope that I will be able to do this for my final coursework.

I ought to have spent more time on my project over the easter holidays. At the time, I failed to recognize how the time I had available to work on the project would drop off as transfer exams loomed and my time and energy transitioned to being focused on revision. That being said, it is possible that this break prevented me from feeling the burnout that I mentioned in the evaluation of my previous project. As such, it may have meant that I was actually more effective than I would have been had I not taken the break.